

Models

 Copy ▾

Qwen3.5 - How to Run Locally

Run the new Qwen3.5 LLMs including Medium: Qwen3.5-35B-A3B, 27B, 122B-A10B, Small: Qwen3.5-0.8B, 2B, 4B, 9B and 397B-A17B on your local device!

Qwen3.5 is Alibaba's new model family, including Qwen3.5-**35B**-A3B, **27B**, **122B**-A10B and **397B**-A17B and the new **Small** series: Qwen3.5-0.8B, 2B, 4B and 9B. The multimodal hybrid reasoning LLMs deliver the strongest performances for their sizes. They support **256K context** across 201 languages, have **thinking** + **non-thinking**, and excel in agentic coding, vision, chat, and long-context tasks. The 35B and 27B models work on a 22GB Mac / RAM device. See all [GGUFs here ↗](#).

Run Qwen3.5 Tutorials

Fine-tune Qwen3.5

✓ **Mar 17 Update:** You can now run Qwen3.5 in [Unisloth Studio](#).

Mar 5 Update: Redownload Qwen3.5-**35B**, **27B**, **122B** and **397B**.

- All GGUFs now updated with an **improved quantization** algorithm.
- All use our **new imatrix data**. See some improvements in chat, coding, long context, and tool-calling use-cases.
- **Tool-calling improved** following our chat template fixes. **Fix is universal** and applies to **any** Qwen3.5 format and **any** uploader.
- **Check new GGUF benchmarks for Unisloth performance results + our MXFP4 investigation.**
- We're retiring MXFP4 layers from 3 Qwen3.5 GGUFs: Q2_K_XL, Q3_K_XL and Q4_K_XL.

All uploads use Unsloth [Dynamic 2.0](#) for SOTA quantization performance - so 4-bit has important layers upcasted to 8 or 16-bit. Thank you Qwen for providing Unsloth with day zero access. You can also [fine-tune Qwen3.5](#) with Unsloth.

 To enable or disable thinking see [How to enable or disable reasoning & thinking](#). Qwen3.5 Small models disables by default.

Usage Guide

Table: Inference hardware requirements (units = total memory: RAM + VRAM, or unified memory)

 Search

Qwen3.5	3-bit	4-bit	6-bit	8-bit	BF16
0.8B + 2B	3 GB	3.5 GB	5 GB	7.5 GB	9 GB
4B	4.5 GB	5.5 GB	7 GB	10 GB	14 GB
9B	5.5 GB	6.5 GB	9 GB	13 GB	19 GB
27B	14 GB	17 GB	24 GB	30 GB	54 GB
35B-A3B	17 GB	22 GB	30 GB	38 GB	70 GB
122B-A10B	60 GB	70 GB	106 GB	132 GB	245 GB
397B-A17B	180 GB	214 GB	340 GB	512 GB	810 GB

 For best performance, make sure your total available memory (VRAM + system RAM) exceeds the size of the quantized model file you're downloading. If it doesn't, llama.cpp can still run via SSD/HDD offloading, but inference will be slower.

Between **27B** and **35B-A3B**, use 27B if you want slightly more accurate results and can't fit in your device. Go for 35B-A3B if you want much faster inference.

Recommended Settings

- **Maximum context window:** 262,144 (can be extended to 1M via YaRN)
- `presence_penalty = 0.0 to 2.0` default this is off, but to reduce repetitions, you can use this, however using a higher value may result in **slight decrease in performance**
- **Adequate Output Length:** 32,768 tokens for most queries

 If you're getting gibberish, your context length might be set too low. Or try using `--cache-type-k bf16 --cache-type-v bf16` which might help.

As Qwen3.5 is hybrid reasoning, thinking and non-thinking mode have different settings:

Thinking mode:

General tasks	Precise coding tasks (e.g. WebDev)
temperature = 1.0	temperature = 0.6
top_p = 0.95	top_p = 0.95
top_k = 20	top_k = 20
min_p = 0.0	min_p = 0.0
presence_penalty = 1.5	presence_penalty = 0.0
repeat_penalty = disabled or 1.0	repeat_penalty = disabled or 1.0

Thinking mode for general tasks:

```
temperature=1.0, top_p=0.95, top_k=20, min_p=0.0,  
presence_penalty=1.5, repetition_penalty=1.0
```

Thinking mode for precise coding tasks:

```
temperature=0.6, top_p=0.95, top_k=20, min_p=0.0,  
presence_penalty=0.0, repetition_penalty=1.0
```

Instruct (non-thinking) mode settings:

General tasks	Reasoning tasks
temperature = 0.7	temperature = 1.0
top_p = 0.8	top_p = 0.95
top_k = 20	top_k = 20
min_p = 0.0	min_p = 0.0
presence_penalty = 1.5	presence_penalty = 1.5
repeat_penalty = disabled or 1.0	repeat_penalty = disabled or 1.0

⚠ To **disable thinking / reasoning**, use `--chat-template-kwargs`
`'{"enable_thinking":false}'`

If you're on **Windows Powershell**, use: `--chat-template-kwargs "`
`{"enable_thinking\":false}"`

Use 'true' and 'false' interchangeably.

For Qwen3.5 0.8B, 2B, 4B and 9B, reasoning is disabled by default. To enable it, use: `--chat-template-kwargs '{"enable_thinking":true}'`

Instruct (non-thinking) for general tasks:

```
temperature=0.7, top_p=0.8, top_k=20, min_p=0.0,
presence_penalty=1.5, repetition_penalty=1.0
```

Instruct (non-thinking) for reasoning tasks:

```
temperature=1.0, top_p=0.95, top_k=20, min_p=0.0,
presence_penalty=1.5, repetition_penalty=1.0
```

Qwen3.5 Inference Tutorials:

Because Qwen3.5 comes in many different sizes, we'll be using Dynamic 4-bit `MXFP4_M0E` GGUF variants for all inference workloads. Click below to navigate to designated model instructions:

Run in Unsloth Studio

Qwen3.5-35B-A3B

27B

122B-A10B

397B-A17B

Small (0.8B - 9B)

Unsloth Dynamic GGUF uploads:

Qwen3.5-35B-A3B ↗	Qwen3.5-27B ↗	Qwen3.5-122B-A10B ↗	Qwen3.5-397B-A17B ↗
Qwen3.5-0.8B ↗	Qwen3.5-2B ↗	Qwen3.5-4B ↗	Qwen3.5-9B ↗

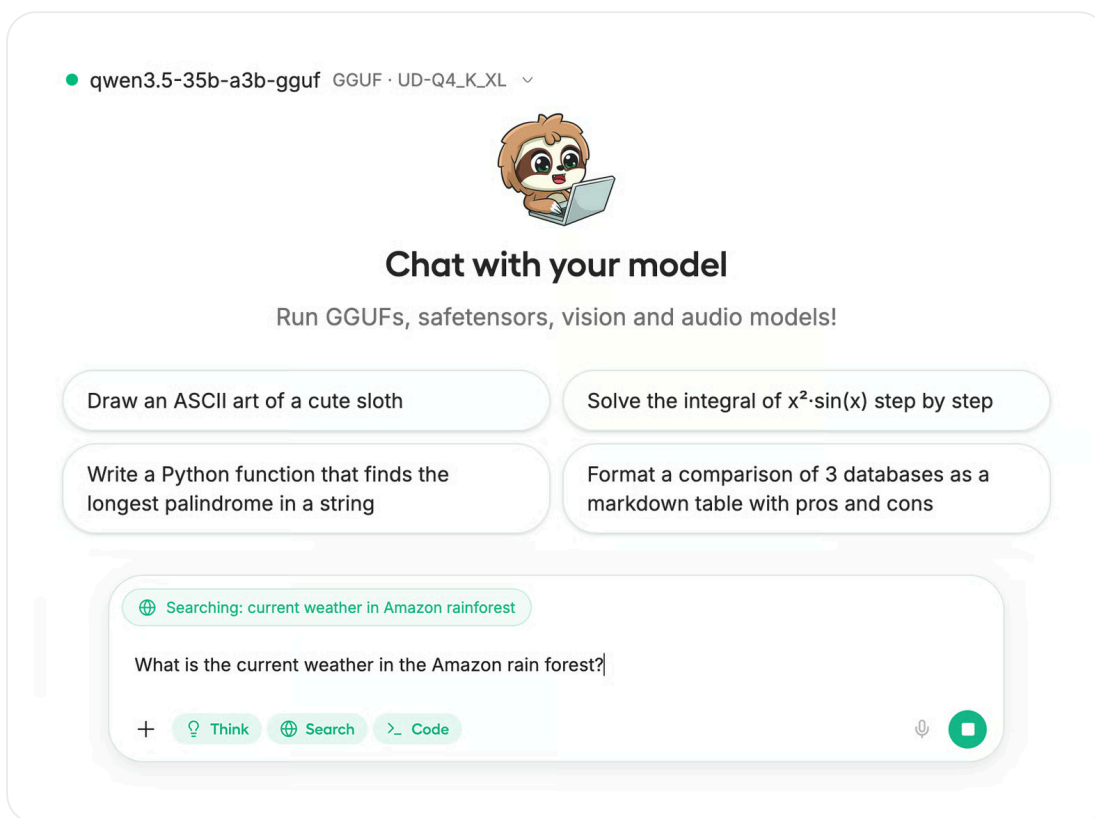
 `presence_penalty = 0.0 to 2.0` default this is off, but to reduce repetitions, you can use this, however using a higher value may result in **slight decrease in performance**.

Currently no Qwen3.5 GGUF works in Ollama due to separate mmproj vision files. Use llama.cpp compatible backends.

Unsloth Studio Guide

Qwen3.5 can be run and fine-tuned in [Unsloth Studio](#), our new open-source web UI for local AI. Unsloth Studio lets you run models locally on **MacOS**, **Windows**, Linux and:

- Search, download, [run GGUFs](#) and safetensor models
- [Self-healing tool calling](#) + **web search**
- [Code execution](#) (Python, Bash)
- [Automatic inference](#) parameter tuning (temp, top-p, etc.)
- Fast CPU + GPU inference via llama.cpp
- [Train LLMs](#) 2x faster with 70% less VRAM



1 Install Unsloth

Run in your terminal:

MacOS, Linux, WSL:

```
curl -fsSL https://unsloth.ai/install.sh | sh
```

Windows PowerShell:

```
irm https://unsloth.ai/install.ps1 | iex
```

✓ Installation will be quick and take approx 1-2 mins.

2 Launch Unsloth

MacOS, Linux, WSL and Windows:

```
unsloth studio -H 0.0.0.0 -p 8888
```

```
..unsloth_studio) >> unsloth studio -H 0.0.0.0 -p 8888
Launching with studio venv...
Matplotlib is building the font cache; this may take a moment.
INFO: Frontend loaded from /Users/matthew/test/.unsloth_studio/lib/python3.12/site-packages/studio/frontend/dist
INFO: Started server process [16622]
INFO: Waiting for application startup.
Hardware detected: CPU (no GPU backend available)

=====
DEFAULT ADMIN ACCOUNT CREATED
Sign in with the seeded credentials and change the password immediately:

username: unsloth
password:

=====

INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
(timestamp": "2025-03-17T04:55:15.684073Z", "level": "Info", "event": "Pre-caching helper GGUF: unsloth/Qwen3-4B-Instruct-2507-GGUF/Qwe
n3-4B-Instruct-2507-UD-Q4_K_XL.gguf")

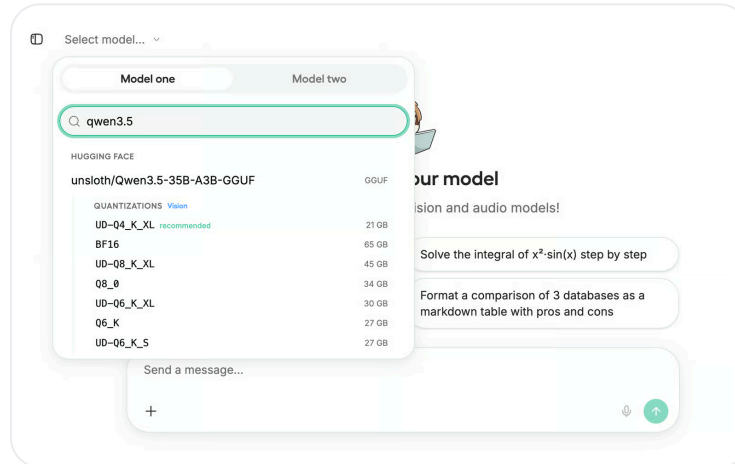
=====
🚀 Unsloth Studio is running on port 8000
Local: http://localhost:8000
External: http://2600:1702:6b10:e770:2c97:b972:404d:b035:8000
API: http://2600:1702:6b10:e770:2c97:b972:404d:b035:8000/api
Health: http://2600:1702:6b10:e770:2c97:b972:404d:b035:8000/api/health
=====
```

Then open `http://localhost:8888` in your browser.

3

Search and download Qwen3.5

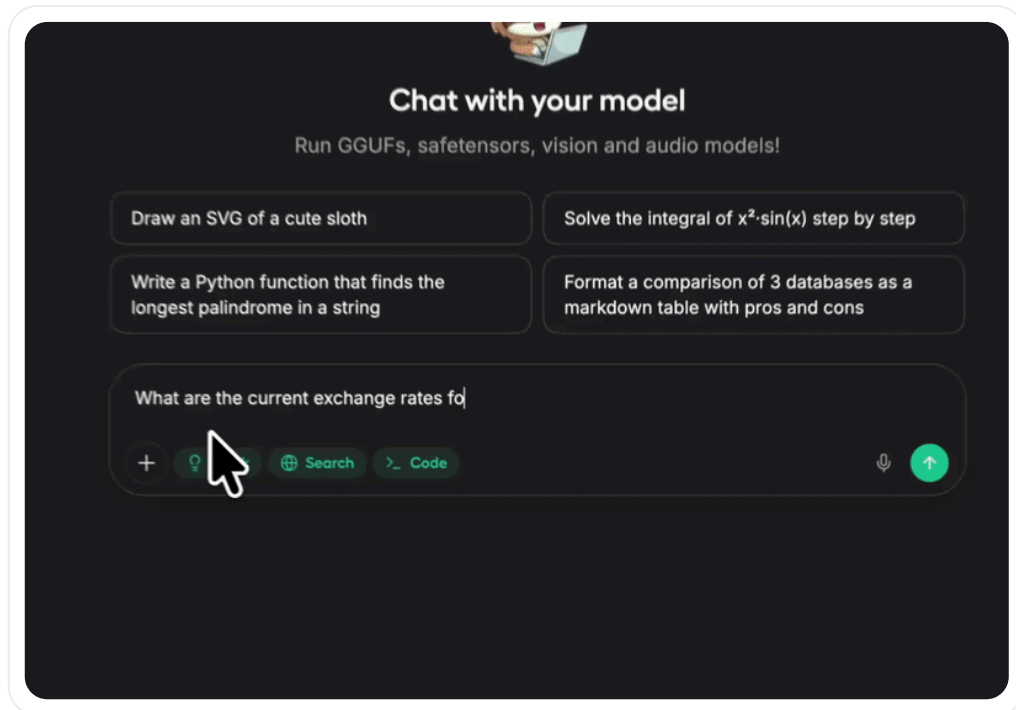
On first launch you will need to create a password to secure your account and sign in again later. Then go to the [Unslloth Chat](#) tab and search for Qwen3.5 in the search bar and download your desired model and quant.



4 Run Qwen3.5

Inference parameters should be auto-set when using Unsloth Studio, however you can still change it manually. You can also edit the context length, chat template and other settings.

For more information, you can view our [Unsloth Studio inference guide](#).



Llama.cpp Guides

Qwen3.5-35B-A3B

For this guide we will be utilizing Dynamic 4-bit which works great on a 24GB RAM / Mac device for fast inference. Because the model is only around 72GB at full F16 precision, we won't need to worry much about performance.

GGUF: [Qwen3.5-35B-A3B-GGUF](#) ↗

For these tutorials, we will using [llama.cpp](#) ↗ for fast local inference, especially if you have a CPU.

- 1 Obtain the latest `llama.cpp` on [GitHub here ↗](#). You can follow the build instructions below as well. Change `-DGGML_CUDA=ON` to `-DGGML_CUDA=OFF` if you don't have a GPU or just want CPU inference. **For Apple Mac / Metal devices**, set `-DGGML_CUDA=OFF` then continue as usual - Metal support is on by default.

```
apt-get update
apt-get install pciutils build-essential cmake curl
libcurl4-openssl-dev -y
git clone https://github.com/ggml-org/llama.cpp
cmake llama.cpp -B llama.cpp/build \
    -DBUILD_SHARED_LIBS=OFF -DGGML_CUDA=ON
cmake --build llama.cpp/build --config Release -j --clean-
first --target llama-cli llama-mtmd-cli llama-server llama-
gguf-split
cp llama.cpp/build/bin/llama-* llama.cpp
```

2

If you want to use `llama.cpp` directly to load models, you can do the below: (:Q4_K_M) is the quantization type. You can also download via Hugging Face (point 3). This is similar to `ollama run`. Use `export LLAMA_CACHE="folder"` to force `llama.cpp` to save to a specific location. The model has a maximum of 256K context length.

Follow one of the specific commands below, according to your use-case:

Thinking mode:

Precise coding tasks (e.g. WebDev):

```
export LLAMA_CACHE="unsloth/Qwen3.5-35B-A3B-GGUF"
./llama.cpp/llama-cli \
  -hf unsloth/Qwen3.5-35B-A3B-GGUF:UD-Q4_K_XL \
  --temp 0.6 \
  --top-p 0.95 \
  --top-k 20 \
  --min-p 0.00
```

General tasks:

```
export LLAMA_CACHE="unsloth/Qwen3.5-35B-A3B-GGUF"
./llama.cpp/llama-cli \
  -hf unsloth/Qwen3.5-35B-A3B-GGUF:UD-Q4_K_XL \
  --temp 1.0 \
  --top-p 0.95 \
  --top-k 20 \
  --min-p 0.00
```

Non-thinking mode:

General tasks:

```
export LLAMA_CACHE="unsloth/Qwen3.5-35B-A3B-GGUF"
./llama.cpp/llama-server \
  -hf unsloth/Qwen3.5-35B-A3B-GGUF:UD-Q4_K_XL \
  --temp 0.7 \
  --top-p 0.8 \
  --top-k 20 \
  --min-p 0.00 \
  --chat-template-kwarg '{"enable_thinking":false}'
```

Reasoning tasks:

```
export LLAMA_CACHE="unsloth/Qwen3.5-35B-A3B-GGUF"
./llama.cpp/llama-server \
  -hf unsloth/Qwen3.5-35B-A3B-GGUF:UD-Q4_K_XL \
  --temp 1.0 \
  --top-p 0.95 \
  --top-k 20 \
  --min-p 0.00 \
  --chat-template-kwarg '{"enable_thinking":false}'
```

3

Download the model via (after installing `pip install huggingface_hub hf_transfer`). You can choose Q4_K_M or other quantized versions like `UD-Q4_K_XL`. We recommend using at least 2-bit dynamic quant `UD-Q2_K_XL` to balance size and accuracy. If downloads get stuck, see: [Hugging Face Hub, XET debugging](#)

```
hf download unsloth/Qwen3.5-35B-A3B-GGUF \
  --local-dir unsloth/Qwen3.5-35B-A3B-GGUF \
  --include "*mmproj-F16*" \
  --include "*UD-Q4_K_XL*" # Use "*UD-Q2_K_XL*" for
Dynamic 2bit
```

4 Then run the model in conversation mode:

```
./llama.cpp/llama-cli \  
  --model unsloth/Qwen3.5-35B-A3B-GGUF/Qwen3.5-35B-A3B-UD-  
Q4_K_XL.gguf \  
  --mmproj unsloth/Qwen3.5-35B-A3B-GGUF/mmproj-F16.gguf \  
  --temp 1.0 \  
  --top-p 0.95 \  
  --min-p 0.00 \  
  --top-k 20
```

Qwen3.5 Small (0.8B • 2B • 4B • 9B)

⚠ **For Qwen3.5 0.8B, 2B, 4B and 9B, reasoning is disabled by default.** To enable it, use: `--chat-template-kwags '{"enable_thinking":true}'`

On Windows use: `--chat-template-kwags "{\\"enable_thinking\\":true}"`

For the Qwen3.5 Small series, because they're so small, all you need to do is change the model name in the scripts to desired variant. For this specific guide we'll be using the 9B parameter variant. To run them all in near full precision, you'll just need 12GB of RAM / VRAM / unified memory device. GGUFs:

[Qwen3.5-0.8B ↗](#)

[Qwen3.5-2B ↗](#)

[Qwen3.5-4B ↗](#)

[Qwen3.5-9B ↗](#)

1

Obtain the latest `llama.cpp` on [GitHub here](#) ↗. You can follow the build instructions below as well. Change `-DGGML_CUDA=ON` to `-DGGML_CUDA=OFF` if you don't have a GPU or just want CPU inference.

```
apt-get update
apt-get install pciutils build-essential cmake curl
libcurl4-openssl-dev -y
git clone https://github.com/ggml-org/llama.cpp
cmake llama.cpp -B llama.cpp/build \
    -DBUILD_SHARED_LIBS=OFF -DGGML_CUDA=ON
cmake --build llama.cpp/build --config Release -j --clean-
first --target llama-cli llama-mtmd-cli llama-server llama-
gguf-split
cp llama.cpp/build/bin/llama-* llama.cpp
```

2

If you want to use `llama.cpp` directly to load models, you can do the below: (:Q4_K_XL) is the quantization type. You can also download via Hugging Face (point 3). This is similar to `ollama run`. Use `export LLAMA_CACHE="folder"` to force `llama.cpp` to save to a specific location. The model has a maximum of 256K context length.

Follow one of the specific commands below, according to your use-case:



To use another variant other than 9B, you can change the '9B' to: 0.8B, 2B or 4B etc.

Thinking mode (disabled by default)



Qwen3.5 Small models disable thinking by default. Use llama-server to enable it.

```
export LLAMA_CACHE="unsloth/Qwen3.5-9B-GGUF"
./llama.cpp/llama-server \
  -hf unsloth/Qwen3.5-9B-GGUF:UD-Q4_K_XL \
  --temp 0.6 \
  --top-p 0.95 \
  --top-k 20 \
  --min-p 0.00 \
  --alias "unsloth/Qwen3.5-9B-GGUF" \
  --port 8001 \
  --chat-template-kwarg '{"enable_thinking":true}'
```

General tasks:

```
export LLAMA_CACHE="unsloth/Qwen3.5-9B-GGUF"
./llama.cpp/llama-server \
  -hf unsloth/Qwen3.5-9B-GGUF:UD-Q4_K_XL \
  --temp 1.0 \
  --top-p 0.95 \
  --top-k 20 \
  --min-p 0.00 \
  --alias "unsloth/Qwen3.5-9B-GGUF" \
  --port 8001 \
  --chat-template-kwarg '{"enable_thinking":true}'
```

- ✔ To use another variant other than 9B, you can change the '9B' to: 0.8B, 2B or 4B etc.

Non-thinking mode is already on by default

General tasks:

```
export LLAMA_CACHE="unsloth/Qwen3.5-9B-GGUF"
./llama.cpp/llama-cli \
  -hf unsloth/Qwen3.5-9B-GGUF:UD-Q4_K_XL \
  --temp 0.7 \
  --top-p 0.8 \
  --top-k 20 \
  --min-p 0.00
```

Reasoning tasks:

```
export LLAMA_CACHE="unsloth/Qwen3.5-9B-GGUF"
./llama.cpp/llama-cli \
  -hf unsloth/Qwen3.5-9B-GGUF:UD-Q4_K_XL \
  --temp 1.0 \
  --top-p 0.95 \
  --top-k 20 \
  --min-p 0.00
```

- 3 Download the model via (after installing `pip install huggingface_hub hf_transfer`). You can choose Q4_K_M or other quantized versions like `UD-Q4_K_XL` . We recommend using at least 2-bit dynamic quant `UD-Q2_K_XL` to balance size and accuracy. If downloads get stuck, see: [Hugging Face Hub, XET debugging](#)

```
hf download unsloth/Qwen3.5-9B-GGUF \
  --local-dir unsloth/Qwen3.5-9B-GGUF \
  --include "*mmpoj-F16*" \
  --include "*UD-Q4_K_XL*" # Use "*UD-Q2_K_XL*" for
Dynamic 2bit
```

- 4 Then run the model in conversation mode:

```
./llama.cpp/llama-cli \  
    --model unsloth/Qwen3.5-9B-GGUF/Qwen3.5-9B-UD-  
Q4_K_XL.gguf \  
    --mmproj unsloth/Qwen3.5-9B-GGUF/mmproj-F16.gguf \  
    --temp 1.0 \  
    --top-p 0.95 \  
    --min-p 0.00 \  
    --top-k 20
```

Qwen3.5-27B

For this guide we will be utilizing Dynamic 4-bit which works great on a 18GB RAM / Mac device for fast inference. GGUF: [Qwen3.5-27B-GGUF ↗](#)

- 1 Obtain the latest `llama.cpp` [on GitHub here ↗](#). You can follow the build instructions below as well. Change `-DGGML_CUDA=ON` to `-DGGML_CUDA=OFF` if you don't have a GPU or just want CPU inference.

```
apt-get update  
apt-get install pciutils build-essential cmake curl  
libcurl4-openssl-dev -y  
git clone https://github.com/ggml-org/llama.cpp  
cmake llama.cpp -B llama.cpp/build \  
    -DBUILD_SHARED_LIBS=OFF -DGGML_CUDA=ON  
cmake --build llama.cpp/build --config Release -j --clean-  
first --target llama-cli llama-mtmd-cli llama-server llama-  
gguf-split  
cp llama.cpp/build/bin/llama-* llama.cpp
```

2

If you want to use `llama.cpp` directly to load models, you can do the below: (:Q4_K_M) is the quantization type. You can also download via Hugging Face (point 3). This is similar to `ollama run` . Use `export LLAMA_CACHE="folder"` to force `llama.cpp` to save to a specific location. The model has a maximum of 256K context length.

Follow one of the specific commands below, according to your use-case:

Thinking mode:

Precise coding tasks (e.g. WebDev):

```
export LLAMA_CACHE="unsloth/Qwen3.5-27B-GGUF"
./llama.cpp/llama-cli \
  -hf unsloth/Qwen3.5-27B-GGUF:UD-Q4_K_XL \
  --temp 0.6 \
  --top-p 0.95 \
  --top-k 20 \
  --min-p 0.00
```

General tasks:

```
export LLAMA_CACHE="unsloth/Qwen3.5-27B-GGUF"
./llama.cpp/llama-cli \
  -hf unsloth/Qwen3.5-27B-GGUF:UD-Q4_K_XL \
  --temp 1.0 \
  --top-p 0.95 \
  --top-k 20 \
  --min-p 0.00
```

Non-thinking mode:

General tasks:

```
export LLAMA_CACHE="unsloth/Qwen3.5-27B-GGUF"
./llama.cpp/llama-server \
  -hf unsloth/Qwen3.5-27B-GGUF:UD-Q4_K_XL \
  --temp 0.7 \
  --top-p 0.8 \
  --top-k 20 \
  --min-p 0.00 \
  --chat-template-kwarg '{"enable_thinking":false}'
```

Reasoning tasks:

```
export LLAMA_CACHE="unsloth/Qwen3.5-27B-GGUF"
./llama.cpp/llama-server \
  -hf unsloth/Qwen3.5-27B-GGUF:UD-Q4_K_XL \
  --temp 1.0 \
  --top-p 0.95 \
  --top-k 20 \
  --min-p 0.00 \
  --chat-template-kwarg '{"enable_thinking":false}'
```

- 3 Download the model via (after installing `pip install huggingface_hub hf_transfer`). You can choose `MXFP4_MOE` or other quantized versions like `UD-Q4_K_XL` . We recommend using at least 2-bit dynamic quant `UD-Q2_K_XL` to balance size and accuracy. If downloads get stuck, see: [Hugging Face Hub, XET debugging](#)

```
hf download unsloth/Qwen3.5-27B-GGUF \
  --local-dir unsloth/Qwen3.5-27B-GGUF \
  --include "*mmproj-F16*" \
  --include "*UD-Q4_K_XL*" # Use "*UD-Q2_K_XL*" for
Dynamic 2bit
```

- 4 Then run the model in conversation mode:

```
./llama.cpp/llama-cli \  
  --model unsloth/Qwen3.5-27B-GGUF/Qwen3.5-27B-UD-  
Q4_K_XL.gguf \  
  --mmproj unsloth/Qwen3.5-27B-GGUF/mmproj-F16.gguf \  
  --temp 1.0 \  
  --top-p 0.95 \  
  --min-p 0.00 \  
  --top-k 20
```

Qwen3.5-122B-A10B

For this guide we will be utilizing Dynamic 4-bit which works great on a 70GB RAM / Mac device for fast inference. GGUF: [Qwen3.5-122B-A10B-GGUF ↗](#)

- 1 Obtain the latest `llama.cpp` on [GitHub here ↗](#). You can follow the build instructions below as well. Change `-DGGML_CUDA=ON` to `-DGGML_CUDA=OFF` if you don't have a GPU or just want CPU inference.

```
apt-get update  
apt-get install pciutils build-essential cmake curl  
libcurl4-openssl-dev -y  
git clone https://github.com/ggml-org/llama.cpp  
cmake llama.cpp -B llama.cpp/build \  
  -DBUILD_SHARED_LIBS=OFF -DGGML_CUDA=ON  
cmake --build llama.cpp/build --config Release -j --clean-  
first --target llama-cli llama-mtmd-cli llama-server llama-  
gguf-split  
cp llama.cpp/build/bin/llama-* llama.cpp
```

2

If you want to use `llama.cpp` directly to load models, you can do the below: (:Q4_K_M) is the quantization type. You can also download via Hugging Face (point 3). This is similar to `ollama run`. Use `export LLAMA_CACHE="folder"` to force `llama.cpp` to save to a specific location. The model has a maximum of 256K context length.

Follow one of the specific commands below, according to your use-case:

Thinking mode:

Precise coding tasks (e.g. WebDev):

```
export LLAMA_CACHE="unsloth/Qwen3.5-122B-A10B-GGUF"
./llama.cpp/llama-cli \
  -hf unsloth/Qwen3.5-122B-A10B-GGUF:UD-Q4_K_XL \
  --temp 0.6 \
  --top-p 0.95 \
  --top-k 20 \
  --min-p 0.00
```

General tasks:

```
export LLAMA_CACHE="unsloth/Qwen3.5-122B-A10B-GGUF"
./llama.cpp/llama-cli \
  -hf unsloth/Qwen3.5-122B-A10B-GGUF:UD-Q4_K_XL \
  --temp 1.0 \
  --top-p 0.95 \
  --top-k 20 \
  --min-p 0.00
```

Non-thinking mode:

General tasks:

```
export LLAMA_CACHE="unsloth/Qwen3.5-122B-A10B-GGUF"
./llama.cpp/llama-server \
  -hf unsloth/Qwen3.5-122B-A10B-GGUF:UD-Q4_K_XL \
  --temp 0.7 \
  --top-p 0.8 \
  --top-k 20 \
  --min-p 0.00 \
  --chat-template-kwarg '{"enable_thinking":false}'
```

Reasoning tasks:

```
export LLAMA_CACHE="unsloth/Qwen3.5-122B-A10B-GGUF"
./llama.cpp/llama-server \
  -hf unsloth/Qwen3.5-122B-A10B-GGUF:UD-Q4_K_XL \
  --temp 1.0 \
  --top-p 0.95 \
  --top-k 20 \
  --min-p 0.00 \
  --chat-template-kwarg '{"enable_thinking":false}'
```

3

Download the model via (after installing `pip install huggingface_hub hf_transfer`). You can choose `MXFP4_M0E` (dynamic 4bit) or other quantized versions like `UD-Q4_K_XL`. We recommend using at least 2-bit dynamic quant `UD-Q2_K_XL` to balance size and accuracy. If downloads get stuck, see: [Hugging Face Hub, XET debugging](#)

```
hf download unsloth/Qwen3.5-122B-A10B-GGUF \
  --local-dir unsloth/Qwen3.5-122B-A10B-GGUF \
  --include "*mmproj-F16*" \
  --include "*UD-Q4_K_XL*" # Use "*UD-Q2_K_XL*" for
Dynamic 2bit
```

4 Then run the model in conversation mode:

```
./llama.cpp/llama-cli \  
  --model unsloth/Qwen3.5-122B-A10B-GGUF/UD-  
Q4_K_XL/Qwen3.5-122B-A10B-UD-Q4_K_XL-00001-of-00003.gguf \  
  --mmproj unsloth/Qwen3.5-122B-A10B-GGUF/mmproj-F16.gguf \  
\  
  --temp 0.6 \  
  --top-p 0.95 \  
  --top-k 20 \  
  --min-p 0.00
```

Qwen3.5-397B-A17B

Qwen3.5-397B-A17B is in the same performance tier as Gemini 3 Pro, Claude Opus 4.5, and GPT-5.2. The full 397B checkpoint is ~807GB on disk, but via [Unsloth's 397B GGUFs](#) ↗ you can run:

- **3-bit:** fits on **192GB RAM** systems (e.g., a 192GB Mac)
- **4-bit (MXFP4):** fits on **256GB RAM**. Unsloth **4-bit dynamic UD-Q4_K_XL** is **~214GB on disk** - loads directly on a **256GB M3 Ultra**
- Runs on a **single 24GB GPU + 256GB system RAM** via **MoE offloading**, reaching **25+ tokens/s**
- **8-bit** needs **~512GB RAM/VRAM**

 See [397B quantization benchmarks](#) on how Unsloth GGUFs perform.

1

Obtain the latest `llama.cpp` on [GitHub here](#) ↗. You can follow the build instructions below as well. Change `-DGGML_CUDA=ON` to `-DGGML_CUDA=OFF` if you don't have a GPU or just want CPU inference.

```
apt-get update
apt-get install pciutils build-essential cmake curl
libcurl4-openssl-dev -y
git clone https://github.com/ggml-org/llama.cpp
cmake llama.cpp -B llama.cpp/build \
    -DBUILD_SHARED_LIBS=OFF -DGGML_CUDA=ON
cmake --build llama.cpp/build --config Release -j --clean-
first --target llama-cli llama-mtmd-cli llama-server llama-
gguf-split
cp llama.cpp/build/bin/llama-* llama.cpp
```

- 2 If you want to use `llama.cpp` directly to load models, you can do the below: (:Q4_K_M) is the quantization type. You can also download via Hugging Face (point 3). This is similar to `ollama run`. Use `export LLAMA_CACHE="folder"` to force `llama.cpp` to save to a specific location. Remember the model has only a maximum of 256K context length.

Follow this for **thinking** mode:

```
export LLAMA_CACHE="unsloth/Qwen3.5-397B-A17B-GGUF"
./llama.cpp/llama-cli \
  -hf unsloth/Qwen3.5-397B-A17B-GGUF:UD-Q4_K_XL \
  --temp 0.6 \
  --top-p 0.95 \
  --top-k 20 \
  --min-p 0.00
```

Follow this for **non-thinking** mode:

```
export LLAMA_CACHE="unsloth/Qwen3.5-397B-A17B-GGUF"
./llama.cpp/llama-server \
  -hf unsloth/Qwen3.5-397B-A17B-GGUF:UD-Q4_K_XL \
  --temp 0.7 \
  --top-p 0.8 \
  --top-k 20 \
  --min-p 0.00 \
  --chat-template-kwarg '{"enable_thinking":false}'
```

- 3 Download the model via (after installing `pip install huggingface_hub hf_transfer`). You can choose `MXFP4_MOE` (dynamic 4bit) or other quantized versions like `UD-Q4_K_XL`. We recommend using at least 2-bit dynamic quant `UD-Q2_K_XL` to balance size and accuracy. If downloads get stuck, see: [Hugging Face Hub, XET debugging](#)

```
hf download unsloth/Qwen3.5-397B-A17B-GGUF \
  --local-dir unsloth/Qwen3.5-397B-A17B-GGUF \
  --include "*mmproj-F16*" \
  --include "*UD-Q4_K_XL" # Use "*UD-Q2_K_XL*" for Dynamic
2bit
```

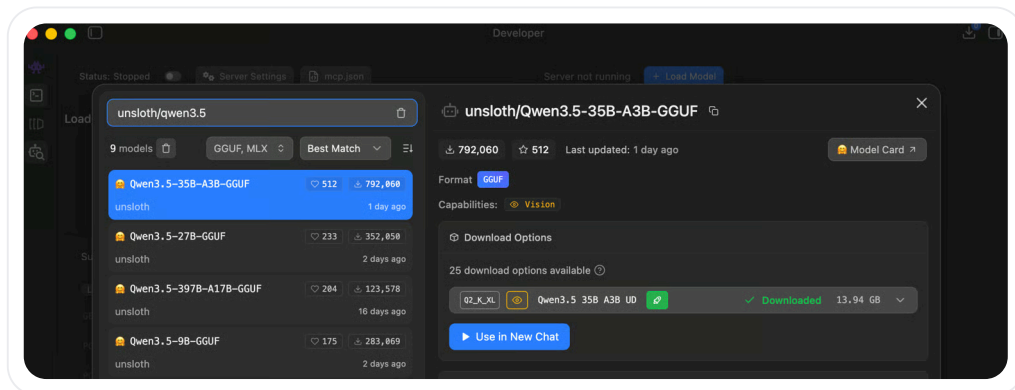
- 4 You can edit `--threads 32` for the number of CPU threads, `--n-gpu-layers 2` for GPU offloading on how many layers. Try adjusting it if your GPU goes out of memory. Also remove it if you have CPU only inference.

```
./llama.cpp/llama-cli \  
  --model unsloth/Qwen3.5-397B-A17B-GGUF/UD-  
Q4_K_XL/Qwen3.5-397B-A17B-UD-Q4_K_XL-00001-of-00006.gguf \  
  --mmproj unsloth/Qwen3.5-397B-A17B-GGUF/mmproj-F16.gguf \  
\  
  --temp 0.6 \  
  --top-p 0.95 \  
  --top-k 20 \  
  --min-p 0.00
```

🧠 LM Studio Guide

For this guide, we'll be using [LM Studio](#), a unified UI interface for running LLMs. The '💡 Thinking' and 'Non-thinking' toggle may not appear by default so we'll need some extra steps to get it working.

- 1 Download [LM Studio](#) for your device. Then open Model Search, search for 'unsloth/qwen3.5', and download the GGUF (quant) that you desire.



- 2 **Thinking Toggle instructions:** After downloading, Open your Terminal / PowerShell and try: `lms --help`. Then if LM Studio appears normally with many commands, run:

```
lms get unsloth/qwen3.5-4b
```

This will get a yaml file which enables your GGUF to have the '💡 Thinking' and 'Non-thinking' toggle appear. You can change `4b` to the desired quant you'd like to have.

```
michaelhan — -zsh — 80x6
[michaelhan@Michaels-MacBook-Pro ~ % lms get unsloth/qwen3.5-4b
✓ Satisfied unsloth/qwen3.5-4b
└─ ✓ Satisfied Qwen3.5 4B UD Q4_K_XL [GGUF]
michaelhan@Michaels-MacBook-Pro ~ %
```

Otherwise, you can go to [our LM Studio page ↗](#) and download the specific yaml file.

- 3 Restart LM Studio, then load your downloaded model (with the specific thinking toggle you downloaded). You should now see the Thinking toggle enabled. Don't forget to set the [correct parameters](#).

Now you can toggle think or no think. Just click the '💡 Think' button. Make sure you set the correct params.

+ 🔗 Think Vision 1% User ⌘U Insert ⌘I ↕

Llama-server serving & OpenAI's completion library

To deploy Qwen3.5-397B-A17B for production, we use `llama-server`. In a new terminal say via tmux, deploy the model via:

```
./llama.cpp/llama-server \  
--model unsloth/Qwen3.5-35B-A3B-GGUF/Qwen3.5-35B-A3B-UD-  
Q4_K_XL.gguf \  
  --mmproj unsloth/Qwen3.5-35B-A3B-GGUF/mmproj-F16.gguf \  
  --alias "unsloth/Qwen3.5-35B-A3B" \  
  --temp 0.6 \  
  --top-p 0.95 \  
  --top-k 20 \  
  --min-p 0.00 \  
  --port 8001
```

Then in a new terminal, after doing `pip install openai`, do:

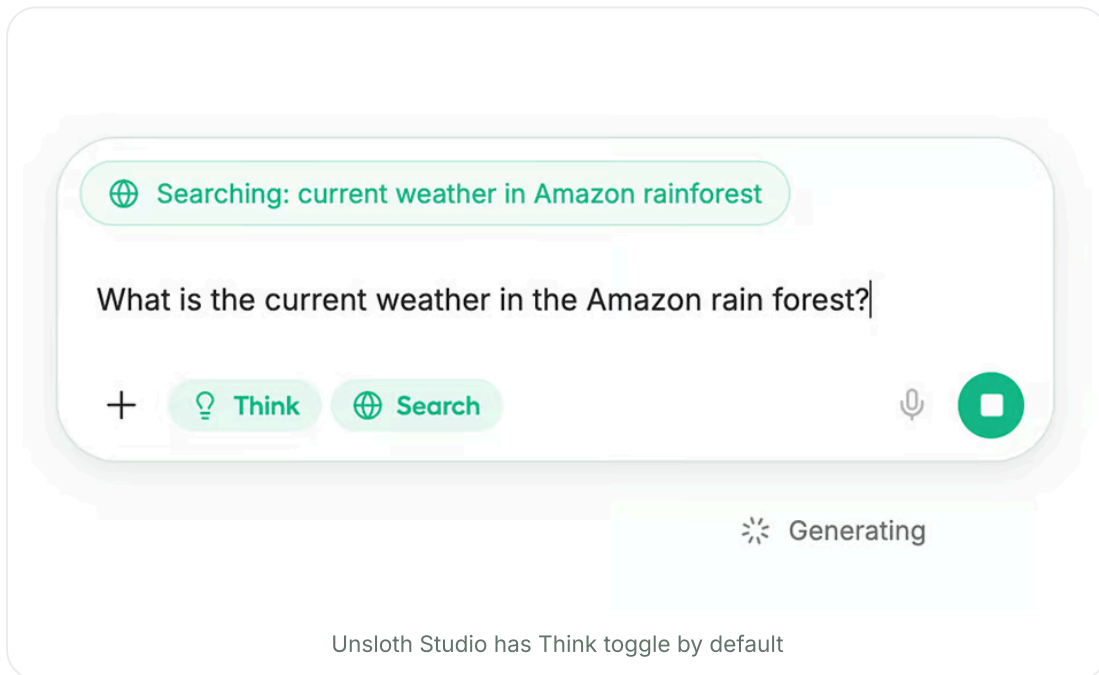
```
from openai import OpenAI  
import json  
openai_client = OpenAI(  
    base_url = "http://127.0.0.1:8001/v1",  
    api_key = "sk-no-key-required",  
)  
completion = openai_client.chat.completions.create(  
    model = "unsloth/Qwen3.5-397B-A17B",  
    messages = [{"role": "user", "content": "Create a Snake  
game."}],  
)  
print(completion.choices[0].message.content)
```

🤔 How to enable or disable reasoning & thinking

For the below commands, you can use 'true' and 'false' interchangeably.

[Unsloth Studio](#) automatically has a 'Think' Toggle for thinking models.

To have Think toggle for LM Studio, [read our guide](#).



❗ To **disable** thinking / reasoning, use within llama-server:

```
--chat-template-kwarg "enable_thinking":false"
```

If you're on **Windows** or Powershell, use: `--chat-template-kwarg "enable_thinking":false"`

❗ To **enable** thinking / reasoning, use within llama-server:

```
--chat-template-kwarg "enable_thinking":true"
```

If you're on **Windows** or Powershell, use: `--chat-template-kwarg "enable_thinking":true"`

 **For Qwen3.5 0.8B, 2B, 4B and 9B, reasoning is disabled by default.** To enable it, use: `--chat-template-kwarg '{"enable_thinking":true}'`

And on Windows or Powershell: `--chat-template-kwarg "{\\"enable_thinking\\":true}"`

As an example for Qwen3.5-9B to enable thinking (default is disabled):

```
./llama.cpp/llama-server \
  --model unsloth/Qwen3.5-9B-GGUF/Qwen3.5-9B-BF16.gguf \
  --alias "unsloth/Qwen3.5-9B-GGUF" \
  --temp 0.6 \
  --top-p 0.95 \
  --top-k 20 \
  --min-p 0.00 \
  --port 8001 \
  --chat-template-kwarg '{"enable_thinking":true}'
```

And then in Python:

```
from openai import OpenAI
import json
openai_client = OpenAI(
    base_url = "http://127.0.0.1:8001/v1",
    api_key = "sk-no-key-required",
)
completion = openai_client.chat.completions.create(
    model = "unsloth/Qwen3.5-9B-GGUF",
    messages = [{"role": "user", "content": "What is 2+2?"},],
)
print(completion.choices[0].message.content)
print(completion.choices[0].message.reasoning_content)
```

```

2 + 2 = 4.
Thinking Process:
1. Analyze the Request: The user is asking "What is 2+2?". This is a basic arithmetic question.
2. Identify the Core Fact: In standard arithmetic, the sum of 2 and 2 is 4.
3. Check for Context/Nuance:
  * Is this a trick question? (e.g., in different bases, like base 2, 2+2 is 10).
  * Is this a philosophical question? (e.g., Gödel, Turing, etc., but usually not implied by a simple query).
  * Is this a test of the AI's capabilities?
  * Most likely, it's a straightforward factual question.
4. Formulate the Answer: Keep it simple and direct. "2+2 equals 4."
5. Review for Safety/Policy: No sensitive or harmful content. Purely mathematical.
6. Final Output Construction: "2 + 2 = 4" or "2+2 is 4."
7. Final Decision: Provide the standard arithmetic answer.

*Self-Correction/Refinement:* While base systems exist, unless specified, the default assumption is base 10. I should just give the standard answer.

*Draft:* 2+2 is 4.

*Final Polish:* 2 + 2 = 4.cw

```

OpenAI Codex & Claude Code

To run the model via local coding agentic workloads, you can [follow our guide](#). Use the `llama-server` we just set up just then, and set the model name to the exact id it reports at `GET /v1/models` (the `--alias` value above, e.g. `unsloth/Qwen3.5-9B-GGUF`). Follow the correct Qwen3.5 parameters and usage instructions.

 Claude Code



 OpenAI Codex



After following the instructions for Claude Code for example you will see:

```

Claude Code v2.1.25

Welcome back Daniel!

unsloth/Qwen3-Coder-Next · API Usage Billing ·

Tips for getting started
Run /init to create a CLAUDE.md file with instructions for Claude

Recent activity
No recent activity

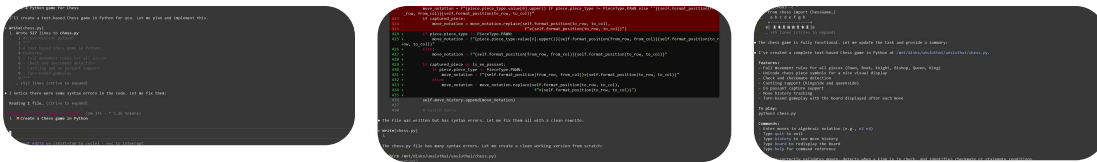
Auth conflict: Both a token (claude.ai) and an API key (ANTHROPIC_API_KEY) are set. This may lead to unexpected behavior.
  • Trying to use claude.ai? Unset the ANTHROPIC_API_KEY environment variable, or claude /logout then say "No" to the API key approval before login.
  • Trying to use ANTHROPIC_API_KEY? claude /logout to sign out of claude.ai.
/model to try Opus 4.5

> |

>> accept edits on (shift+Tab to cycle)

```

We can then ask say `Create a Python game for Chess` :



Tool Calling with Qwen3.5

See [Tool Calling Guide](#) for more details on how to do tool calling. In a new terminal (if using tmux, use CTRL+B+D), we create some tools like adding 2 numbers, executing Python code, executing Linux functions and much more:

```
import json, subprocess, random
from typing import Any
def add_number(a: float | str, b: float | str) -> float:
    return float(a) + float(b)
def multiply_number(a: float | str, b: float | str) -> float:
    return float(a) * float(b)
def subtract_number(a: float | str, b: float | str) -> float:
    return float(a) - float(b)
def write_a_story() -> str:
    return random.choice([
        "A long time ago in a galaxy far far away...",
```

We then use the below functions (copy and paste and execute) which will parse the function calls automatically and call the OpenAI endpoint for any model:

```
from openai import OpenAI
def unsloth_inference(
    messages,
    temperature = 0.6,
    top_p = 0.95,
    top_k = 20,
    min_p = 0.00,
    repetition_penalty = 1.0,
):
    messages = messages.copy()
    openai_client = OpenAI(
```

After launching Qwen3.5 via `llama-server` like in [Qwen3.5](#) or see [Tool Calling Guide](#) for more details, we then can do some tool calls.



We updated Qwen3.5-35B Unsloth Dynamic quants **being SOTA** on nearly all bits. We did over 150 KL Divergence benchmarks, totally **9TB of GGUFs**. We uploaded all research artifacts. We also fixed a **tool calling** chat template **bug** (affects all quant uploaders)

- [illegible]

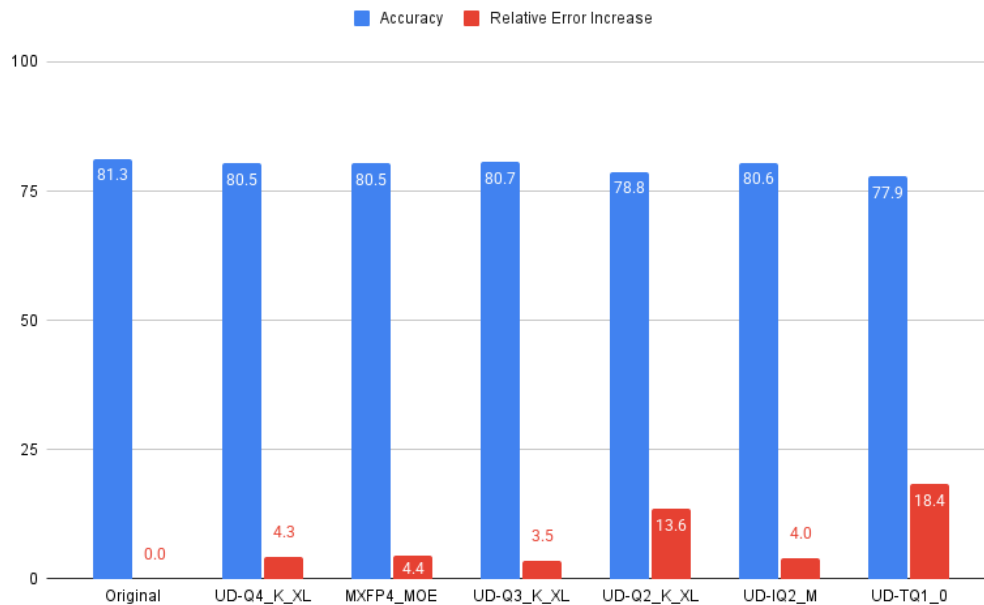
READ OUR DETAILED QWEN3.5 ANALYSIS + BENCHMARKS HERE:



Qwen3.5-397B-A17B Benchmarks

Qwen3.5-397B-A17B: GGUF Evaluation

Benchmark: 750 prompts from Live Code Bench (v6), MMLU Pro, GPQA, and Math500



[Benjamin Marie \(third-party\) benchmarked](#) [Qwen3.5-397B-A17B](#) using Unsloth GGUFs on a **750-prompt mixed suite** (LiveCodeBench v6, MMLU Pro, GPQA, Math500), reporting both **overall accuracy** and **relative error increase** (how much more often the quantized model makes mistakes vs. the original).

Key results (accuracy; change vs. original; relative error increase):

- **Original weights: 81.3%**
- **UD-Q4_K_XL: 80.5%** (*-0.8 points; +4.3% relative error increase*)
- **UD-Q3_K_XL: 80.7%** (*-0.6 points; +3.5% relative error increase*)

`UD-Q4_K_XL` and `UD-Q3_K_XL` stay extremely close to the original, **well under a 1-point accuracy drop** on this suite, which Ben insinuates that you can **sharply reduce memory footprint (~500 GB less)** with little to no practical loss on the tested tasks.

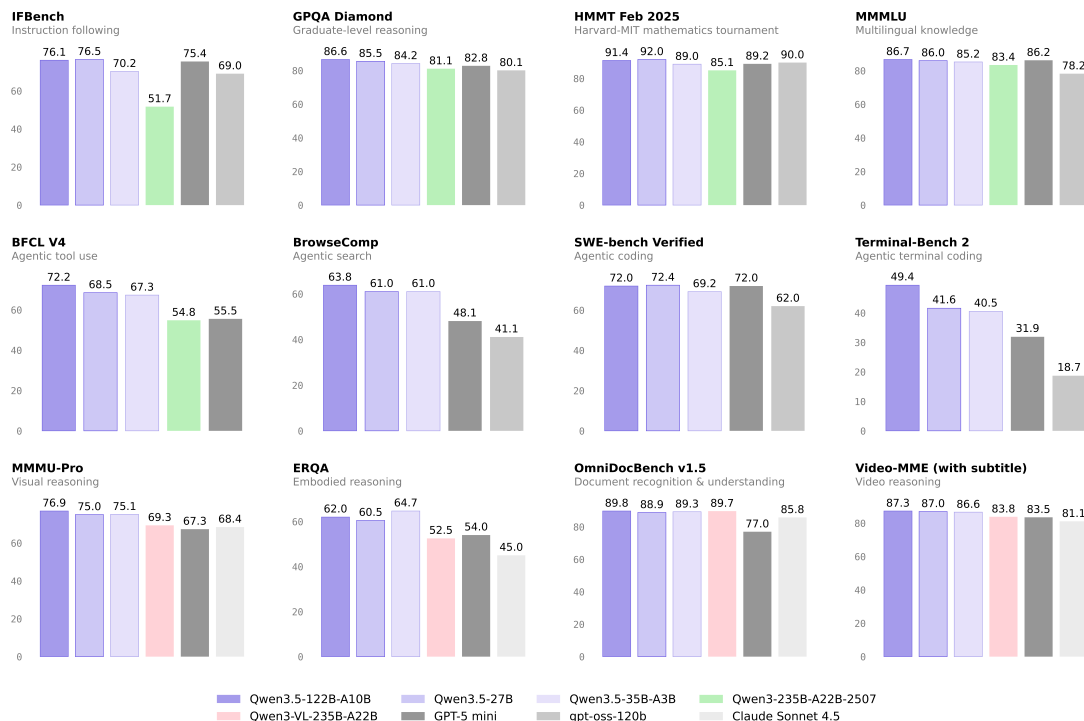
How to choose: Q3 scoring slightly higher than Q4 here is completely plausible as normal run-to-run variance at this scale, so treat **Q3 and Q4 as effectively similar quality** in this benchmark:

- Pick **Q3** if you want the **smallest footprint / best memory savings**
- Pick **Q4** if you want a **slightly more conservative** option with **similar** results

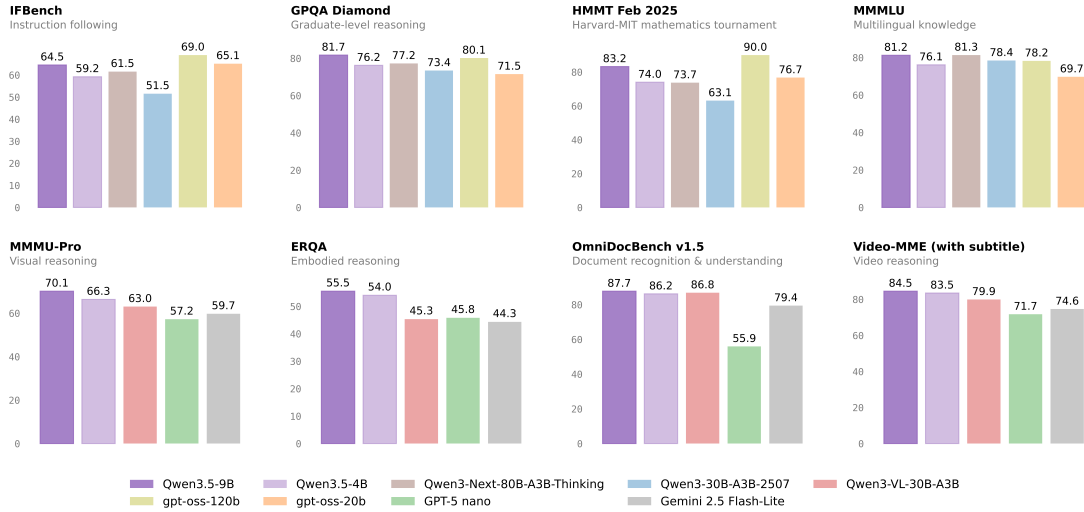
All listed quants utilize our dynamic methodology. Even `UD-IQ2_M` uses a the same methodology of dynamic however the conversion process is different to `UD-Q2-K-XL` where K-XL is usually faster than `UD-IQ2_M` even though it's bigger, so that is why `UD-IQ2_M` may perform better than `UD-Q2-K-XL`.

Official Qwen Benchmarks

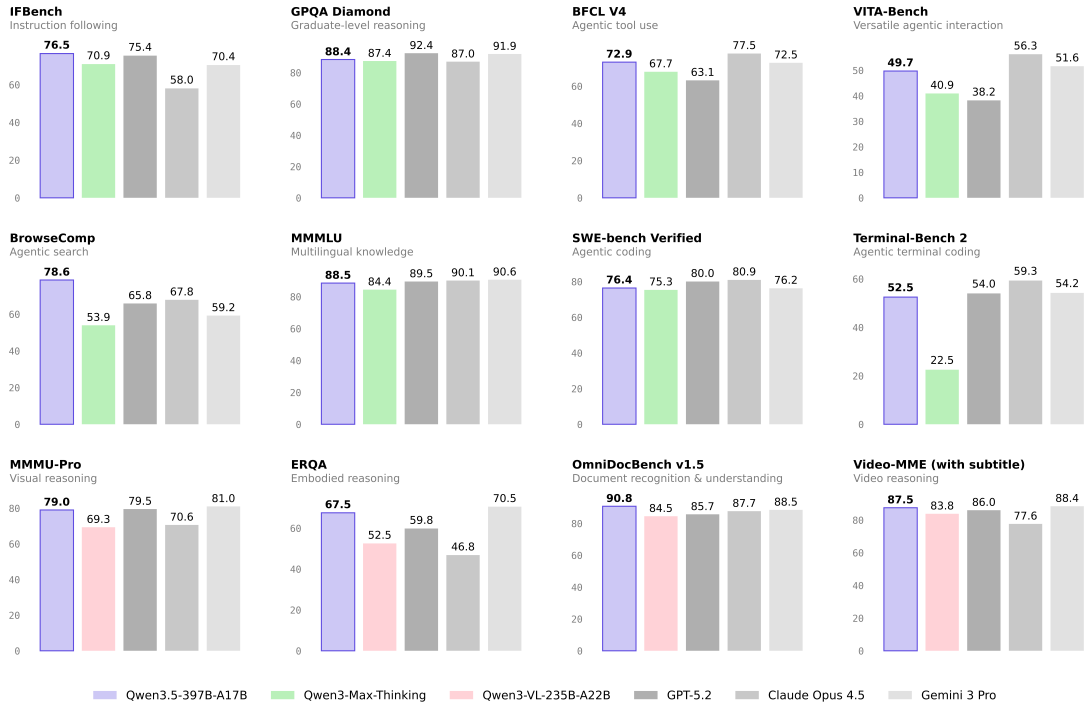
Qwen3.5-35B-A3B, 27B and 122B-A10B Benchmarks



Qwen3.5-4B and 9B Benchmarks

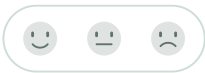


Qwen3.5-397B-A17B Benchmarks



Previous
DiffusionGemma

Next
Fine-tune Qwen3.5



Community

[Reddit r/unsloth](#)

[Twitter \(X\)](#)

[LinkedIn](#)

Resources

[Tutorials](#)

[Docker](#)

[Hugging Face](#)

Company

[Unsloth Studio](#)

[Contact](#)

[Events](#)

